

En el presente informe se documenta la creación de las entidades Documento, DocumentoUsuario y DocumentoUsuariold

La entidad documento corresponde a la tabla Documento descrita en la BD la cual muestra únicamente la información relevante para un documento

CREATE TABLE documentos

```
(
id_documento BIGINT AUTO_INCREMENT PRIMARY KEY,
titulo VARCHAR(255) NOT NULL,
contenido LONGTEXT,
tipo ENUM('EDITABLE', 'ARCHIVO')NOT NULL,
url_archivo VARCHAR(500),
estado ENUM('BORRADOR', 'EN_REVISION', 'APROBADO') DEFAULT 'BORRADOR',
fecha_creacion TIMESTAMP NOT NULL, -- DEFAULT CURRENT_TIMESTAMP,
fecha_actualizacion TIMESTAMP NOT NULL-- DEFAULT CURRENT_TIMESTAMP ON
UPDATE CURRENT_TIMESTAMP
);
```

De allí se es necesario describir este modelo mediante una clase que se aloja en la carpeta models representando en el sistema esta tabla como un objeto

```
package com.example.demo.models;

import jakarta.persistence.*;
import lombok.AllArgsConstructor;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;

import java.time.LocalDateTime;

@Getter //genera los getters automaticamente con lombok
@Setter //genera los setters automaticamente con lombok
@NoArgsConstructor // Genera el constructor vacío (que pide JPA)
@AllArgsConstructor // Genera un constructor con todos los atributos
@Entity
@Table(name = "documentos")
public class Documento {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "id_documento")
    private Long idDocumento;
    @Column(nullable = false, length = 255)
    private String titulo;
    @Column(columnDefinition = "LONGTEXT")
    private String contenido;
    @Enumerated(EnumType.STRING)
    @Column(nullable = false)
    private TipoDocumento tipo;
    private String urlArchivo;
    @Enumerated(EnumType.STRING)
    private EstadoDocumento estado= EstadoDocumento.BORRADOR; // Valor por defecto
```

```

    @Column(name = "fecha_creacion", nullable = false, updatable = false) //La
    fecha de creacion no debe cambiar nunca
    private LocalDateTime fechaCreacion;

    @Column(name = "fecha_actualizacion", nullable = false)
    private LocalDateTime fechaActualizacion;

//METODOS DE AUDITORÍA AUTOMÁTICA
//Ganchos del ciclo de vida
    @PrePersist //anotación JPA que indica que se debe ejecutar automáticamente
    apenas la entidad se guarde (persista) en la bd
    protected void onCreate() {
        fechaCreacion = LocalDateTime.now();
        fechaActualizacion = LocalDateTime.now();
    }

    @PreUpdate //anotación JPA que indica que se debe ejecutar automáticamente
    justo después de que la entidad se actualice en la bd
    protected void onUpdate() {
        fechaActualizacion = LocalDateTime.now(); // Se sobrescribe con la hora
    actual del cambio
    }
//esto también lo puede hacer mysql
}

```

Los tipos de datos enum se representan como clases internas anidadas ya que en el momento solo se realiza el acceso a estos datos en la clase documento. Estas clases pueden trabajar también de forma global desacoplando las y definiéndose en la carpeta models como clases independientes.

A partir de esto se ha creado su respectivo repositorio el cual permite interactuar con los datos de la BD sin realizar explícitamente las consultas descritas en el código.

```

package com.example.demo.repositories;
import com.example.demo.models.EstadoDocumento;
import com.example.demo.models.TipoDocumento;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
import com.example.demo.models.Documento;
import java.util.List;
import java.util.Optional;

@Repository
public interface DocumentoRepository extends JpaRepository<Documento, Long> {

    // El findById(Long id) ya existe por defecto, no hace falta escribirlo.

    // busca por coincidencia exacta de título
    Optional<Documento> findByTitulo(String titulo);
    // buscar por título que contenga una palabra (buscador tipo google)
    List<Documento> findByTituloContainingIgnoreCase(String palabraClave);
}

```

```

// buscar todos los documentos de un mismo estado (ejemplo EN_REVISION)
List<Documento> findByEstado(EstadoDocumento estado);

// buscar por tipo y estado al mismo tiempo
List<Documento> findByTipoAndEstado(TipoDocumento tipo, EstadoDocumento
estado);

// contar cuántos documentos hay de un tipo específico
Long countByTipo(TipoDocumento tipo);
}

```

De igual manera se es necesario representar la relacion entre un documento y usuarios que tiene diferentes roles en ese documento, esto soluciona la cardinalidad del documento y el usuario. La tabla intermedia DocumentoUsuario permite que un documento esté vinculado a muchos usuarios, cada uno con una responsabilidad distinta representado en los atributos de la relacion de un usuario con el rol sobre el documento.

El Rol (Autor, Editor, Supervisor) no es una propiedad del Usuario (un usuario no "se crea" siendo editor de todo) ni es una propiedad del Documento (un documento no "es" un autor).

El Rol es una propiedad de la interacción entre ambos.

Ejemplo: Juan es Autor del Documento A, pero es Supervisor del Documento B.

```

CREATE TABLE documentos_usuarios
(
    id_documento BIGINT,
    id_usuario BIGINT,
    rol_documento ENUM('AUTOR', 'EDITOR', 'SUPERVISOR') NOT NULL, -- no son los
mismos roles que los de las tablas de rol y permisos
    -- supervisor de documento es diferente a supervisor global del sistema
    -- un usuario puede ser autor
    PRIMARY KEY(id_documento, id_usuario),
    FOREIGN KEY (id_documento) REFERENCES documentos(id_documento),
    FOREIGN KEY (id_usuario) REFERENCES usuarios(id_usuario)
);

```

```

package com.example.demo.models;

import jakarta.persistence.*;
import lombok.AllArgsConstructor;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;

@Getter //genera los getters automaticamente con lombok
@Setter //genera los setters automaticamente con lombok

```

```

@NoArgsConstructor // Genera el constructor vacío (que pide JPA)
@AllArgsConstructor // Genera un constructor con todos los atributos
@Entity
@Table(name = "documentos_usuario")
public class DocumentoUsuario {
    @EmbeddedId
    private DocumentoUsuarioId id;
    @ManyToOne
    @MapsId
    @JoinColumn(name = "id_documento")
    private Documento documento;
    @ManyToOne
    @MapsId
    @JoinColumn(name = "id_usuario")
    private Usuario usuario;

    @Enumerated(EnumType.STRING)
    @Column(name = "rol_documento", nullable = false)
    private RolDocumento rolDocumento;
}

```

Se optó por el uso de `@Embeddable` y `@EmbeddedId` para cumplir con el estándar de JPA en el manejo de llaves primarias compuestas. Esta arquitectura permite encapsular los atributos que definen la identidad de la relación (`id_usuario`, `id_documento`) en una clase independiente, promoviendo la cohesión del código y permitiendo que Spring Data JPA gestione de forma nativa la navegación de propiedades en las consultas derivadas del repositorio.

```

package com.example.demo.models;

import jakarta.persistence.Embeddable;
import lombok.AllArgsConstructor;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;

import java.io.Serializable;
import java.util.Objects;

@Getter //genera los getters automaticamente con lombok
@Setter //genera los setters automaticamente con lombok
@NoArgsConstructor // Genera el constructor vacío (que pide JPA)
@AllArgsConstructor // Genera un constructor con todos los atributos
@Embeddable
/*JPA necesita que las llaves compuestas sean serializables para manejar el
cache de segundo nivel con mas largo plazo
*/
public class DocumentoUsuarioId implements Serializable {
    private static final long serialVersionUID = 1L; //encargado de version
    de clase, si se cambia algo lanza excepcion, no fallo
    private Long idDocumento;
    private Long idUsuario;
}

```

```

    /*Sirve para comparar dos objetos en memoria y evitar la creacion de
duplicados*/
    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof DocumentoUsuarioId)) return false;
        DocumentoUsuarioId that = (DocumentoUsuarioId) o;
        return Objects.equals(idDocumento, that.idDocumento) &&
            Objects.equals(idUsuario, that.idUsuario);
    }
    /*Sirve para generar un numero unico basado en los dos ID para organizar
los objetos en sus mapas internos que corresponde
cache del primer nivel haciendo mas eficiente el codigo*/
    @Override
    public int hashCode() {
        return Objects.hash(idDocumento, idUsuario);
    }
}

```

Se ha implementado una tabla asociativa denominada Documentos_Usuarios con el fin de modelar una relación de muchos a muchos (Many-to-Many). Esta decisión de diseño permite que un mismo documento cuente con múltiples usuarios asignados, cada uno desempeñando funciones específicas.

Se ha creado el repositorio DocumentoUsuarioRepository para gestionar la persistencia y consulta de las relaciones entre usuarios y documentos. Este repositorio permite desacoplar la lógica de permisos y roles, facilitando la auditoría de accesos y el filtrado por roles específicos mediante el uso de consultas que combinan la identidad de la relación (Usuario y Documento) con su atributo de rol asignado.

```

package com.example.demo.repositories;

import com.example.demo.models.DocumentoUsuario;
import com.example.demo.models.DocumentoUsuarioId;
import com.example.demo.models.RolDocumento;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import java.util.List;
import java.util.Optional;

@Repository
public interface DocumentoUsuarioRepository extends
JpaRepository<DocumentoUsuario, DocumentoUsuarioId> {
    //buscar todos los documentos de un usuario en especifico
    List<DocumentoUsuario> findById_IdUsuario(Long idUsuario);

    /*Trae los documentos que participan un rol en especifico*/
    List<DocumentoUsuario> findById_IdDocumentoAndRolDocumento(Long
idDocumento, RolDocumento rol);
}

```

```
// Para cambiar el rol de un usuario en un documento rápidamente
// Ejemplo: De EDITOR a SUPERVISOR
Optional<DocumentoUsuario> findById_IdUsuarioAndId_IdDocumento(Long
idUsuario, Long idDocumento);

/*devuelve si un usuario esta asociado a un documento*/
boolean existsById_IdUsuarioAndId_IdDocumento(Long idUsuario, Long
idDocumento);

/*Afirma o deniega la existencia de una relacion, por ejemplo si ya existe
un documento con ciertos usuarios no deberia haber otro*/
boolean existsById_IdUsuarioAndId_IdDocumentoAndRolDocumento(Long
idUsuario, Long idDocumento, RolDocumento rol);
}
```